

자동차IT융합학과 20241268 지윤주

자동차인공지능 이상현교수님

2026/04/26

HW3 : MNIST 숫자 분류와 하이퍼파라미터 분석

0. 기본 condition 코드 구조

본 실험은 28x28 해상도의 MNIST 이미지를 입력받는 다층 퍼셉트론(MLP) 구조를 기반으로 수행하였다. 입력층은 784개의 노드로 구성되며, 은닉층은 기본 128개 노드와 ReLU 활성 함수를 사용하는 단일 계층 구조를 채택하였다. 출력층은 10개의 클래스 분류를 위해 Softmax 연산을 거쳐 최종 확률을 도출하도록 설계하였다. 데이터셋은 Training, Validation, Test 세트로 엄격히 분리하여 실험의 객관성을 확보하였으며, 입력 데이터는 0~1 사이로 정규화하여 학습의 안정성을 높였다. 옵티마이저는 별도의 모멘텀이 없는 SGD를 사용하였고, 손실 함수는 Cross Entropy Loss를 적용하여 총 20 Epoch 동안 학습을 진행하였다. 모든 실험은 변인 통제를 위해 한 번에 하나의 파라미터만 변경하며 수행되었으며, 최종 성능 평가는 검증 세트에서 선정된 최적 조합을 통해 Test Set에서 단 한 번 측정하였다.

*참조 : ipynb 전체 코드 파일 첨부

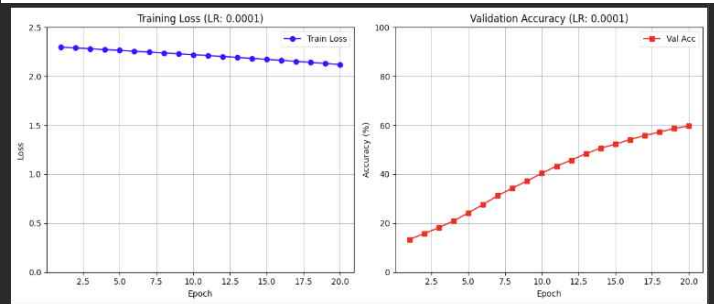
1. lr 조정

lr	최종 loss	최종 val acc	total acc	comment (성능 평가)
0.0001	2.1195	59.68	60.15	최하
0.001	0.5991	85.72	86.81	하
0.01	0.2523	92.32	93.05	중
0.1	0.0516	97.42	97.44	상

학습률이 높을수록(0.1) Global Minimum에 빠르게 접근하여 성능이 극대화되었다. 그래프에서는 학습이 잘 이루어져 Loss가 빠르게 감소하고 accuracy가 100%에 근사하게 접근하는 것을 볼 수 있다. 반면 학습률이 너무 낮은 경우(0.0001) 가중치 업데이트가 미미하여 정해진 시간 내에 충분한 학습이 이루어지지 않는 과소적합(Underfitting) 현상이 관찰되었다. 이 경우 Loss 그래프는 다른 실험값에 비해 한참 상회하며, Accuracy의 경우 60% 밑으로 낮게 나타나는 것을 관찰할 수 있다. 이를 통해 모델의 구조와 데이터셋의 특성에 맞는 적절한 Learning Rate 설정이 학습 효율의 핵심임을 확인하였다.

1.1. lr=0.0001

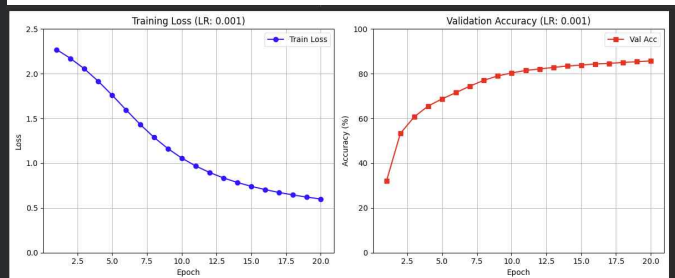
```
Epoch [1/20] Loss: 2.2975, Train Acc: 11.95%, Val Acc: 13.39%
Epoch [2/20] Loss: 2.2893, Train Acc: 14.01%, Val Acc: 15.75%
Epoch [3/20] Loss: 2.2810, Train Acc: 16.51%, Val Acc: 18.19%
Epoch [4/20] Loss: 2.2727, Train Acc: 19.25%, Val Acc: 20.98%
Epoch [5/20] Loss: 2.2643, Train Acc: 22.31%, Val Acc: 24.20%
Epoch [6/20] Loss: 2.2559, Train Acc: 25.66%, Val Acc: 27.65%
Epoch [7/20] Loss: 2.2473, Train Acc: 29.05%, Val Acc: 31.26%
Epoch [8/20] Loss: 2.2386, Train Acc: 32.34%, Val Acc: 34.33%
Epoch [9/20] Loss: 2.2298, Train Acc: 35.44%, Val Acc: 37.29%
Epoch [10/20] Loss: 2.2207, Train Acc: 38.55%, Val Acc: 40.39%
Epoch [11/20] Loss: 2.2116, Train Acc: 41.50%, Val Acc: 43.33%
Epoch [12/20] Loss: 2.2022, Train Acc: 44.26%, Val Acc: 45.75%
Epoch [13/20] Loss: 2.1926, Train Acc: 46.86%, Val Acc: 48.41%
Epoch [14/20] Loss: 2.1828, Train Acc: 49.26%, Val Acc: 50.65%
Epoch [15/20] Loss: 2.1728, Train Acc: 51.35%, Val Acc: 52.26%
Epoch [16/20] Loss: 2.1626, Train Acc: 53.37%, Val Acc: 54.17%
Epoch [17/20] Loss: 2.1522, Train Acc: 55.16%, Val Acc: 55.81%
Epoch [18/20] Loss: 2.1415, Train Acc: 56.70%, Val Acc: 57.22%
Epoch [19/20] Loss: 2.1308, Train Acc: 58.12%, Val Acc: 58.70%
Epoch [20/20] Loss: 2.1195, Train Acc: 59.45%, Val Acc: 59.68%
```



최종 Test Accuracy: 60.15%

1.2. lr=0.001

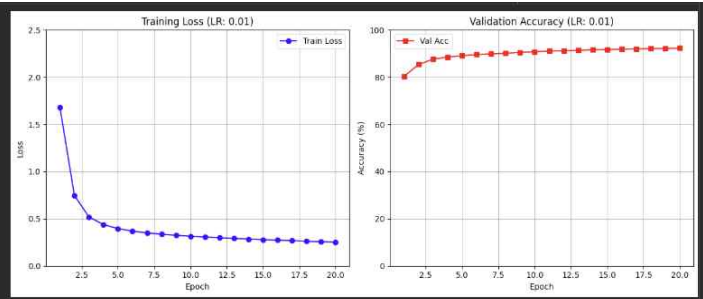
```
Epoch [1/20] Loss: 2.2701, Train Acc: 18.72%, Val Acc: 32.05%
Epoch [2/20] Loss: 2.1711, Train Acc: 44.66%, Val Acc: 53.32%
Epoch [3/20] Loss: 2.0542, Train Acc: 57.60%, Val Acc: 60.75%
Epoch [4/20] Loss: 1.9159, Train Acc: 64.05%, Val Acc: 65.56%
Epoch [5/20] Loss: 1.7600, Train Acc: 67.85%, Val Acc: 68.77%
Epoch [6/20] Loss: 1.5957, Train Acc: 70.67%, Val Acc: 71.57%
Epoch [7/20] Loss: 1.4354, Train Acc: 73.81%, Val Acc: 74.46%
Epoch [8/20] Loss: 1.2895, Train Acc: 76.44%, Val Acc: 77.05%
Epoch [9/20] Loss: 1.1633, Train Acc: 78.70%, Val Acc: 79.00%
Epoch [10/20] Loss: 1.0572, Train Acc: 80.18%, Val Acc: 80.34%
Epoch [11/20] Loss: 0.9690, Train Acc: 81.50%, Val Acc: 81.46%
Epoch [12/20] Loss: 0.8960, Train Acc: 82.48%, Val Acc: 82.13%
Epoch [13/20] Loss: 0.8352, Train Acc: 83.22%, Val Acc: 82.79%
Epoch [14/20] Loss: 0.7844, Train Acc: 83.87%, Val Acc: 83.46%
Epoch [15/20] Loss: 0.7412, Train Acc: 84.44%, Val Acc: 83.84%
Epoch [16/20] Loss: 0.7044, Train Acc: 84.91%, Val Acc: 84.34%
Epoch [17/20] Loss: 0.6726, Train Acc: 85.31%, Val Acc: 84.61%
Epoch [18/20] Loss: 0.6449, Train Acc: 85.61%, Val Acc: 85.02%
Epoch [19/20] Loss: 0.6206, Train Acc: 85.95%, Val Acc: 85.38%
Epoch [20/20] Loss: 0.5991, Train Acc: 86.25%, Val Acc: 85.72%
```



*** 최종 Test Accuracy: 86.81%

1.3. lr=0.01

```
Epoch [1/20] Loss: 1.6821, Train Acc: 64.89%, Val Acc: 80.32%
Epoch [2/20] Loss: 0.7478, Train Acc: 83.82%, Val Acc: 85.39%
Epoch [3/20] Loss: 0.5200, Train Acc: 87.11%, Val Acc: 87.63%
Epoch [4/20] Loss: 0.4384, Train Acc: 88.50%, Val Acc: 88.56%
Epoch [5/20] Loss: 0.3957, Train Acc: 89.29%, Val Acc: 89.08%
Epoch [6/20] Loss: 0.3692, Train Acc: 89.81%, Val Acc: 89.62%
Epoch [7/20] Loss: 0.3507, Train Acc: 90.26%, Val Acc: 89.85%
Epoch [8/20] Loss: 0.3364, Train Acc: 90.59%, Val Acc: 90.15%
Epoch [9/20] Loss: 0.3252, Train Acc: 90.85%, Val Acc: 90.49%
Epoch [10/20] Loss: 0.3154, Train Acc: 91.09%, Val Acc: 90.77%
Epoch [11/20] Loss: 0.3068, Train Acc: 91.32%, Val Acc: 91.10%
Epoch [12/20] Loss: 0.2991, Train Acc: 91.54%, Val Acc: 91.15%
Epoch [13/20] Loss: 0.2920, Train Acc: 91.75%, Val Acc: 91.37%
Epoch [14/20] Loss: 0.2855, Train Acc: 91.90%, Val Acc: 91.63%
Epoch [15/20] Loss: 0.2794, Train Acc: 92.08%, Val Acc: 91.74%
Epoch [16/20] Loss: 0.2735, Train Acc: 92.23%, Val Acc: 91.83%
Epoch [17/20] Loss: 0.2679, Train Acc: 92.43%, Val Acc: 91.99%
Epoch [18/20] Loss: 0.2624, Train Acc: 92.62%, Val Acc: 92.12%
Epoch [19/20] Loss: 0.2571, Train Acc: 92.73%, Val Acc: 92.18%
Epoch [20/20] Loss: 0.2523, Train Acc: 92.91%, Val Acc: 92.32%
```

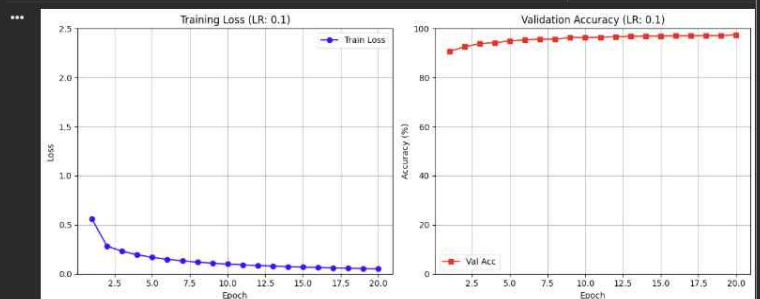


...

최종 Test Accuracy: 93.05%

1.4. lr=0.1

```
Epoch [1/20] Loss: 0.5608, Train Acc: 85.75%, Val Acc: 90.67%
Epoch [2/20] Loss: 0.2838, Train Acc: 91.90%, Val Acc: 92.55%
Epoch [3/20] Loss: 0.2312, Train Acc: 93.40%, Val Acc: 93.84%
Epoch [4/20] Loss: 0.1957, Train Acc: 94.41%, Val Acc: 94.26%
Epoch [5/20] Loss: 0.1693, Train Acc: 95.21%, Val Acc: 94.97%
Epoch [6/20] Loss: 0.1493, Train Acc: 95.78%, Val Acc: 95.38%
Epoch [7/20] Loss: 0.1333, Train Acc: 96.21%, Val Acc: 95.69%
Epoch [8/20] Loss: 0.1201, Train Acc: 96.58%, Val Acc: 95.68%
Epoch [9/20] Loss: 0.1095, Train Acc: 96.94%, Val Acc: 96.31%
Epoch [10/20] Loss: 0.1004, Train Acc: 97.22%, Val Acc: 96.35%
Epoch [11/20] Loss: 0.0928, Train Acc: 97.37%, Val Acc: 96.45%
Epoch [12/20] Loss: 0.0861, Train Acc: 97.59%, Val Acc: 96.69%
Epoch [13/20] Loss: 0.0803, Train Acc: 97.81%, Val Acc: 96.77%
Epoch [14/20] Loss: 0.0744, Train Acc: 97.97%, Val Acc: 96.94%
Epoch [15/20] Loss: 0.0698, Train Acc: 98.10%, Val Acc: 96.95%
Epoch [16/20] Loss: 0.0658, Train Acc: 98.18%, Val Acc: 97.09%
Epoch [17/20] Loss: 0.0615, Train Acc: 98.34%, Val Acc: 97.06%
Epoch [18/20] Loss: 0.0582, Train Acc: 98.41%, Val Acc: 97.18%
Epoch [19/20] Loss: 0.0547, Train Acc: 98.56%, Val Acc: 97.11%
Epoch [20/20] Loss: 0.0516, Train Acc: 98.65%, Val Acc: 97.42%
```



...

최종 Test Accuracy: 97.44%

1. batch size 조정

batchsize	최종 loss	최종 val acc	total acc	comment (성능 평가)
1	0.0008	98.2	98	상 오래걸림
32	0.1342	95.59	95.82	중
100	0.2571	92.62	93.05	중
512	0.4188	88.32	89.81	하

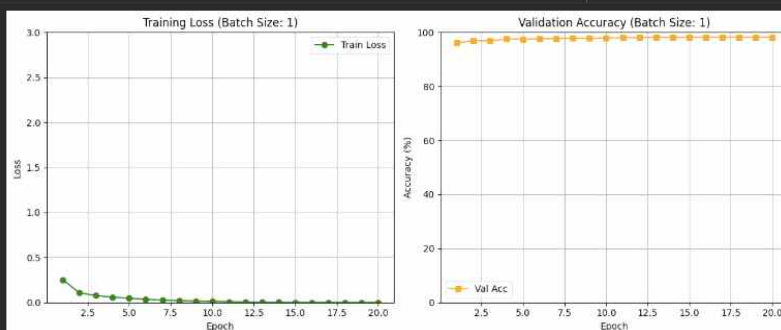
배치 사이즈가 작을수록(Batch 1) 빈번한 가중치 업데이트를 통해 데이터의 세밀한 특징까지 학습하여 높은 정확도를 기록하였다. 하지만 학습 소요 시간을 대폭 증가시키는 단점이 있다. 반대로 배치 사이즈가 큰 경우(Batch 512) 연산 속도는 빠르나, 동일한 epoch 내 업데이트 횟수가 부족하여 학습 초기 단계에 머무르는 경향을 보였다. 결론적으로 정확도와 학습 속도 사이의 균형을 고려했을 때, 본 실험 환경에서는 32~100 사이의 배치 사이즈가 가장 합리적인 선택임을 확인하였다.

2.1. batch size = 1

```

... Epoch [1/20] Loss: 0.2536, Train Acc: 92.44%, Val Acc: 96.01%
Epoch [2/20] Loss: 0.1070, Train Acc: 96.61%, Val Acc: 96.96%
Epoch [3/20] Loss: 0.0772, Train Acc: 97.61%, Val Acc: 96.79%
Epoch [4/20] Loss: 0.0586, Train Acc: 98.17%, Val Acc: 97.51%
Epoch [5/20] Loss: 0.0465, Train Acc: 98.49%, Val Acc: 97.41%
Epoch [6/20] Loss: 0.0368, Train Acc: 98.83%, Val Acc: 97.57%
Epoch [7/20] Loss: 0.0273, Train Acc: 99.13%, Val Acc: 97.70%
Epoch [8/20] Loss: 0.0210, Train Acc: 99.30%, Val Acc: 97.87%
Epoch [9/20] Loss: 0.0149, Train Acc: 99.56%, Val Acc: 97.74%
Epoch [10/20] Loss: 0.0126, Train Acc: 99.64%, Val Acc: 97.90%
Epoch [11/20] Loss: 0.0088, Train Acc: 99.77%, Val Acc: 98.05%
Epoch [12/20] Loss: 0.0059, Train Acc: 99.88%, Val Acc: 98.01%
Epoch [13/20] Loss: 0.0030, Train Acc: 99.98%, Val Acc: 98.16%
Epoch [14/20] Loss: 0.0023, Train Acc: 99.98%, Val Acc: 98.11%
Epoch [15/20] Loss: 0.0016, Train Acc: 100.00%, Val Acc: 98.14%
Epoch [16/20] Loss: 0.0012, Train Acc: 100.00%, Val Acc: 98.19%
Epoch [17/20] Loss: 0.0010, Train Acc: 100.00%, Val Acc: 98.18%
Epoch [18/20] Loss: 0.0009, Train Acc: 100.00%, Val Acc: 98.17%
Epoch [19/20] Loss: 0.0008, Train Acc: 100.00%, Val Acc: 98.19%
Epoch [20/20] Loss: 0.0008, Train Acc: 100.00%, Val Acc: 98.20%

```



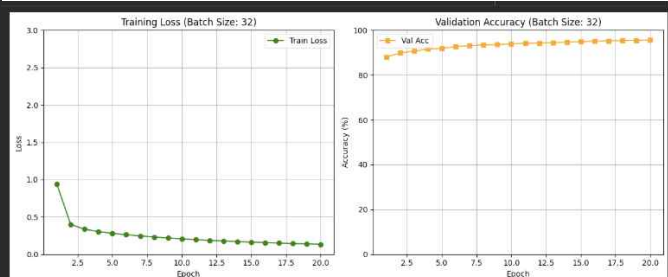
... 최종 Test Accuracy: 98.00%

2.2. batch size = 32

```

... Epoch [1/20] Loss: 0.9435, Train Acc: 79.05%, Val Acc: 87.96%
Epoch [2/20] Loss: 0.3990, Train Acc: 89.11%, Val Acc: 89.88%
Epoch [3/20] Loss: 0.3359, Train Acc: 90.47%, Val Acc: 90.65%
Epoch [4/20] Loss: 0.3045, Train Acc: 91.34%, Val Acc: 91.57%
Epoch [5/20] Loss: 0.2816, Train Acc: 91.99%, Val Acc: 91.89%
Epoch [6/20] Loss: 0.2625, Train Acc: 92.58%, Val Acc: 92.71%
Epoch [7/20] Loss: 0.2458, Train Acc: 93.07%, Val Acc: 93.07%
Epoch [8/20] Loss: 0.2314, Train Acc: 93.51%, Val Acc: 93.40%
Epoch [9/20] Loss: 0.2182, Train Acc: 93.97%, Val Acc: 93.54%
Epoch [10/20] Loss: 0.2069, Train Acc: 94.23%, Val Acc: 93.91%
Epoch [11/20] Loss: 0.1964, Train Acc: 94.52%, Val Acc: 94.14%
Epoch [12/20] Loss: 0.1868, Train Acc: 94.84%, Val Acc: 94.25%
Epoch [13/20] Loss: 0.1785, Train Acc: 95.05%, Val Acc: 94.46%
Epoch [14/20] Loss: 0.1705, Train Acc: 95.31%, Val Acc: 94.61%
Epoch [15/20] Loss: 0.1630, Train Acc: 95.49%, Val Acc: 94.86%
Epoch [16/20] Loss: 0.1566, Train Acc: 95.68%, Val Acc: 95.13%
Epoch [17/20] Loss: 0.1504, Train Acc: 95.88%, Val Acc: 95.22%
Epoch [18/20] Loss: 0.1446, Train Acc: 96.04%, Val Acc: 95.39%
Epoch [19/20] Loss: 0.1392, Train Acc: 96.20%, Val Acc: 95.45%
Epoch [20/20] Loss: 0.1342, Train Acc: 96.35%, Val Acc: 95.59%

```



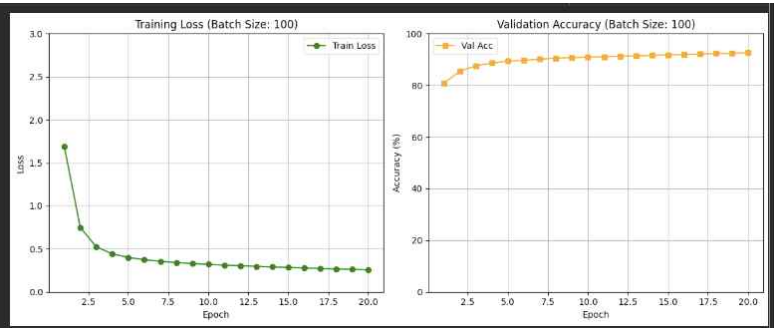
... 최종 Test Accuracy: 95.82%

2.3. batch size = 100

```

Epoch [1/20] Loss: 1.6893, Train Acc: 64.86%, Val Acc: 80.82%
Epoch [2/20] Loss: 0.7502, Train Acc: 83.78%, Val Acc: 85.49%
Epoch [3/20] Loss: 0.5239, Train Acc: 87.14%, Val Acc: 87.56%
Epoch [4/20] Loss: 0.4429, Train Acc: 88.47%, Val Acc: 88.60%
Epoch [5/20] Loss: 0.4008, Train Acc: 89.24%, Val Acc: 89.39%
Epoch [6/20] Loss: 0.3745, Train Acc: 89.78%, Val Acc: 89.67%
Epoch [7/20] Loss: 0.3662, Train Acc: 90.12%, Val Acc: 90.09%
Epoch [8/20] Loss: 0.3421, Train Acc: 90.48%, Val Acc: 90.51%
Epoch [9/20] Loss: 0.3306, Train Acc: 90.70%, Val Acc: 90.69%
Epoch [10/20] Loss: 0.3208, Train Acc: 91.02%, Val Acc: 90.92%
Epoch [11/20] Loss: 0.3126, Train Acc: 91.20%, Val Acc: 91.04%
Epoch [12/20] Loss: 0.3048, Train Acc: 91.44%, Val Acc: 91.25%
Epoch [13/20] Loss: 0.2978, Train Acc: 91.63%, Val Acc: 91.34%
Epoch [14/20] Loss: 0.2912, Train Acc: 91.78%, Val Acc: 91.64%
Epoch [15/20] Loss: 0.2851, Train Acc: 92.00%, Val Acc: 91.79%
Epoch [16/20] Loss: 0.2790, Train Acc: 92.19%, Val Acc: 91.89%
Epoch [17/20] Loss: 0.2731, Train Acc: 92.37%, Val Acc: 92.08%
Epoch [18/20] Loss: 0.2677, Train Acc: 92.53%, Val Acc: 92.30%
Epoch [19/20] Loss: 0.2624, Train Acc: 92.64%, Val Acc: 92.34%
Epoch [20/20] Loss: 0.2571, Train Acc: 92.84%, Val Acc: 92.62%

```



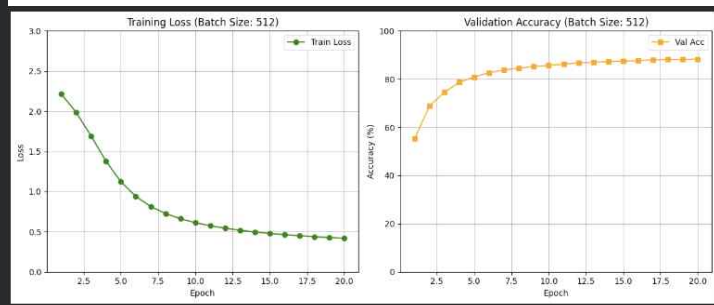
*** 최종 Test Accuracy: 93.05%

2.4. batch size = 512

```

Epoch [1/20] Loss: 2.2140, Train Acc: 30.79%, Val Acc: 55.25%
Epoch [2/20] Loss: 1.9858, Train Acc: 63.44%, Val Acc: 68.82%
Epoch [3/20] Loss: 1.6921, Train Acc: 72.29%, Val Acc: 74.57%
Epoch [4/20] Loss: 1.3903, Train Acc: 77.01%, Val Acc: 78.70%
Epoch [5/20] Loss: 1.1229, Train Acc: 79.97%, Val Acc: 80.88%
Epoch [6/20] Loss: 0.9401, Train Acc: 81.97%, Val Acc: 82.63%
Epoch [7/20] Loss: 0.8145, Train Acc: 83.50%, Val Acc: 83.79%
Epoch [8/20] Loss: 0.7266, Train Acc: 84.55%, Val Acc: 84.53%
Epoch [9/20] Loss: 0.6619, Train Acc: 85.26%, Val Acc: 85.15%
Epoch [10/20] Loss: 0.6129, Train Acc: 85.93%, Val Acc: 85.68%
Epoch [11/20] Loss: 0.5747, Train Acc: 86.46%, Val Acc: 86.24%
Epoch [12/20] Loss: 0.5441, Train Acc: 86.87%, Val Acc: 86.72%
Epoch [13/20] Loss: 0.5183, Train Acc: 87.27%, Val Acc: 87.00%
Epoch [14/20] Loss: 0.4970, Train Acc: 87.59%, Val Acc: 87.25%
Epoch [15/20] Loss: 0.4793, Train Acc: 87.93%, Val Acc: 87.47%
Epoch [16/20] Loss: 0.4636, Train Acc: 88.20%, Val Acc: 87.62%
Epoch [17/20] Loss: 0.4504, Train Acc: 88.38%, Val Acc: 87.97%
Epoch [18/20] Loss: 0.4385, Train Acc: 88.61%, Val Acc: 88.11%
Epoch [19/20] Loss: 0.4280, Train Acc: 88.82%, Val Acc: 88.14%
Epoch [20/20] Loss: 0.4188, Train Acc: 89.00%, Val Acc: 88.32%

```



*** 최종 Test Accuracy: 89.81%

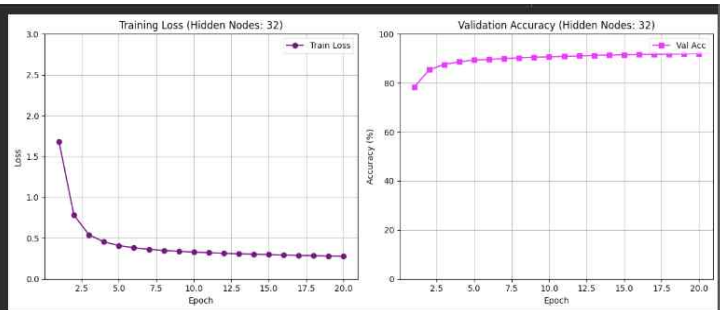
3. hidden node 조정

hiddennode	최종 loss	최종 val acc	total acc	comment (성능 평가)
32	0.2775	91.99	92.31	
64	0.2605	92.26	92.78	
128	0.2484	92.62	93.36	
512	0.2428	92.59	93.43	

은닉층 노드 수에 따른 실험 결과, 노드 수가 증가할수록 모델이 데이터의 복잡한 특징을 효과적으로 학습하여 정확도가 향상됨을 확인하였다. 특히 Node 32 조건에서는 모델의 용량이 제한되어 Node 128 이상의 조건에 비해 학습의 한계치가 낮게 형성되었다. 다만, 노드가 늘어날수록 연산 비용(Time)이 함께 증가하므로, 목적에 맞는 적절한 모델 크기를 설정하는 '트레이드 오프(Trade-off)' 고려가 필요함을 이해하였다.

3.1. hidden node = 32

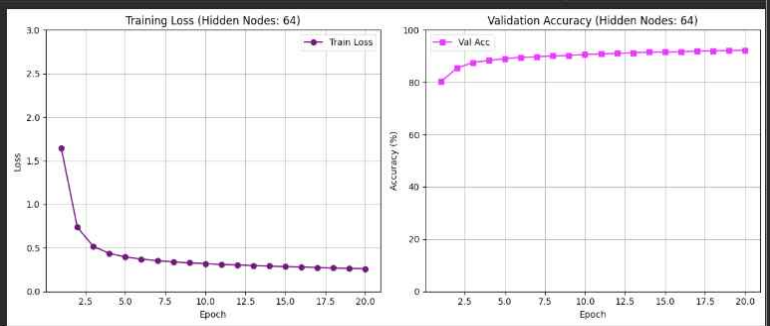
```
Epoch [1/20] Loss: 1.6837, Train Acc: 59.08%, Val Acc: 78.40%
Epoch [2/20] Loss: 0.7819, Train Acc: 82.59%, Val Acc: 85.42%
Epoch [3/20] Loss: 0.5425, Train Acc: 86.37%, Val Acc: 87.57%
Epoch [4/20] Loss: 0.4543, Train Acc: 87.96%, Val Acc: 88.66%
Epoch [5/20] Loss: 0.4095, Train Acc: 88.80%, Val Acc: 89.39%
Epoch [6/20] Loss: 0.3822, Train Acc: 89.37%, Val Acc: 89.64%
Epoch [7/20] Loss: 0.3634, Train Acc: 89.87%, Val Acc: 89.93%
Epoch [8/20] Loss: 0.3495, Train Acc: 90.20%, Val Acc: 90.26%
Epoch [9/20] Loss: 0.3385, Train Acc: 90.42%, Val Acc: 90.53%
Epoch [10/20] Loss: 0.3296, Train Acc: 90.64%, Val Acc: 90.60%
Epoch [11/20] Loss: 0.3217, Train Acc: 90.86%, Val Acc: 90.91%
Epoch [12/20] Loss: 0.3150, Train Acc: 91.05%, Val Acc: 91.06%
Epoch [13/20] Loss: 0.3090, Train Acc: 91.24%, Val Acc: 91.22%
Epoch [14/20] Loss: 0.3033, Train Acc: 91.42%, Val Acc: 91.38%
Epoch [15/20] Loss: 0.2986, Train Acc: 91.52%, Val Acc: 91.55%
Epoch [16/20] Loss: 0.2939, Train Acc: 91.69%, Val Acc: 91.72%
Epoch [17/20] Loss: 0.2894, Train Acc: 91.79%, Val Acc: 91.69%
Epoch [18/20] Loss: 0.2854, Train Acc: 91.90%, Val Acc: 91.79%
Epoch [19/20] Loss: 0.2814, Train Acc: 92.01%, Val Acc: 91.88%
Epoch [20/20] Loss: 0.2775, Train Acc: 92.08%, Val Acc: 91.99%
```



... 최종 Test Accuracy: 92.31%

3.2. hidden node = 64

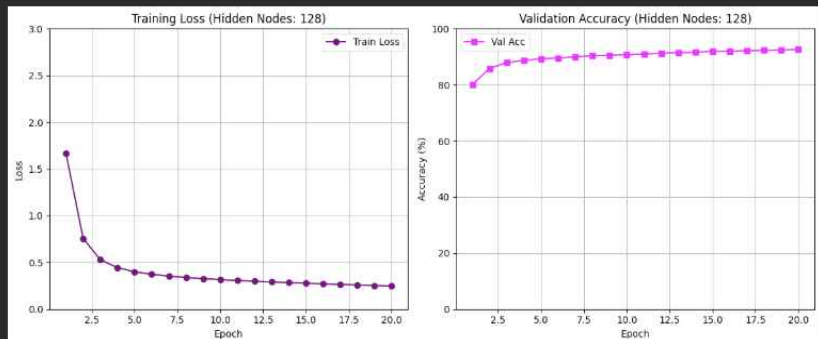
```
Epoch [1/20] Loss: 1.6451, Train Acc: 64.93%, Val Acc: 80.25%
Epoch [2/20] Loss: 0.7376, Train Acc: 83.95%, Val Acc: 85.41%
Epoch [3/20] Loss: 0.5174, Train Acc: 87.12%, Val Acc: 87.58%
Epoch [4/20] Loss: 0.4376, Train Acc: 88.48%, Val Acc: 88.36%
Epoch [5/20] Loss: 0.3964, Train Acc: 89.19%, Val Acc: 89.03%
Epoch [6/20] Loss: 0.3709, Train Acc: 89.69%, Val Acc: 89.52%
Epoch [7/20] Loss: 0.3530, Train Acc: 90.10%, Val Acc: 89.76%
Epoch [8/20] Loss: 0.3393, Train Acc: 90.44%, Val Acc: 90.07%
Epoch [9/20] Loss: 0.3284, Train Acc: 90.69%, Val Acc: 90.28%
Epoch [10/20] Loss: 0.3192, Train Acc: 90.94%, Val Acc: 90.61%
Epoch [11/20] Loss: 0.3112, Train Acc: 91.16%, Val Acc: 90.85%
Epoch [12/20] Loss: 0.3040, Train Acc: 91.44%, Val Acc: 91.07%
Epoch [13/20] Loss: 0.2975, Train Acc: 91.60%, Val Acc: 91.27%
Epoch [14/20] Loss: 0.2913, Train Acc: 91.77%, Val Acc: 91.59%
Epoch [15/20] Loss: 0.2856, Train Acc: 91.96%, Val Acc: 91.59%
Epoch [16/20] Loss: 0.2801, Train Acc: 92.13%, Val Acc: 91.67%
Epoch [17/20] Loss: 0.2750, Train Acc: 92.28%, Val Acc: 91.94%
Epoch [18/20] Loss: 0.2699, Train Acc: 92.46%, Val Acc: 91.97%
Epoch [19/20] Loss: 0.2651, Train Acc: 92.59%, Val Acc: 92.16%
Epoch [20/20] Loss: 0.2605, Train Acc: 92.73%, Val Acc: 92.26%
```



... 최종 Test Accuracy: 92.78%

3.3. hidden node = 128

```
Epoch [1/20] Loss: 1.6693, Train Acc: 63.89%, Val Acc: 80.16%
Epoch [2/20] Loss: 0.7582, Train Acc: 83.17%, Val Acc: 85.84%
Epoch [3/20] Loss: 0.5299, Train Acc: 86.92%, Val Acc: 87.94%
Epoch [4/20] Loss: 0.4450, Train Acc: 88.35%, Val Acc: 88.71%
Epoch [5/20] Loss: 0.4009, Train Acc: 89.16%, Val Acc: 89.29%
Epoch [6/20] Loss: 0.3735, Train Acc: 89.72%, Val Acc: 89.55%
Epoch [7/20] Loss: 0.3541, Train Acc: 90.09%, Val Acc: 89.97%
Epoch [8/20] Loss: 0.3393, Train Acc: 90.49%, Val Acc: 90.42%
Epoch [9/20] Loss: 0.3273, Train Acc: 90.81%, Val Acc: 90.53%
Epoch [10/20] Loss: 0.3170, Train Acc: 91.09%, Val Acc: 90.77%
Epoch [11/20] Loss: 0.3077, Train Acc: 91.37%, Val Acc: 91.00%
Epoch [12/20] Loss: 0.2995, Train Acc: 91.55%, Val Acc: 91.25%
Epoch [13/20] Loss: 0.2918, Train Acc: 91.79%, Val Acc: 91.46%
Epoch [14/20] Loss: 0.2847, Train Acc: 92.00%, Val Acc: 91.60%
Epoch [15/20] Loss: 0.2778, Train Acc: 92.22%, Val Acc: 91.97%
Epoch [16/20] Loss: 0.2715, Train Acc: 92.39%, Val Acc: 91.96%
Epoch [17/20] Loss: 0.2653, Train Acc: 92.62%, Val Acc: 92.15%
Epoch [18/20] Loss: 0.2595, Train Acc: 92.76%, Val Acc: 92.30%
Epoch [19/20] Loss: 0.2538, Train Acc: 92.90%, Val Acc: 92.42%
Epoch [20/20] Loss: 0.2484, Train Acc: 93.03%, Val Acc: 92.62%
```

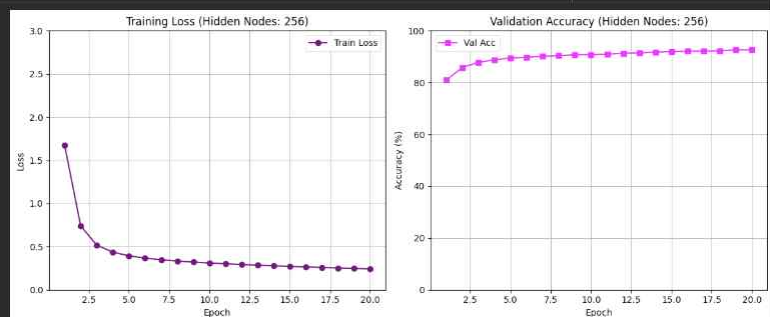


... 최종 Test Accuracy: 93.36%

```
import matplotlib.pyplot as plt
```

3.4. hidden node = 256

```
Epoch [1/20] Loss: 1.6765, Train Acc: 65.33%, Val Acc: 80.94%
Epoch [2/20] Loss: 0.7403, Train Acc: 83.83%, Val Acc: 85.80%
Epoch [3/20] Loss: 0.5174, Train Acc: 87.11%, Val Acc: 87.81%
Epoch [4/20] Loss: 0.4364, Train Acc: 88.50%, Val Acc: 88.77%
Epoch [5/20] Loss: 0.3942, Train Acc: 89.28%, Val Acc: 89.46%
Epoch [6/20] Loss: 0.3676, Train Acc: 89.82%, Val Acc: 89.78%
Epoch [7/20] Loss: 0.3487, Train Acc: 90.26%, Val Acc: 90.15%
Epoch [8/20] Loss: 0.3338, Train Acc: 90.58%, Val Acc: 90.36%
Epoch [9/20] Loss: 0.3217, Train Acc: 90.88%, Val Acc: 90.70%
Epoch [10/20] Loss: 0.3113, Train Acc: 91.22%, Val Acc: 90.78%
Epoch [11/20] Loss: 0.3019, Train Acc: 91.44%, Val Acc: 90.97%
Epoch [12/20] Loss: 0.2934, Train Acc: 91.71%, Val Acc: 91.29%
Epoch [13/20] Loss: 0.2856, Train Acc: 91.94%, Val Acc: 91.43%
Epoch [14/20] Loss: 0.2784, Train Acc: 92.17%, Val Acc: 91.76%
Epoch [15/20] Loss: 0.2716, Train Acc: 92.39%, Val Acc: 91.93%
Epoch [16/20] Loss: 0.2652, Train Acc: 92.58%, Val Acc: 92.06%
Epoch [17/20] Loss: 0.2592, Train Acc: 92.73%, Val Acc: 92.13%
Epoch [18/20] Loss: 0.2536, Train Acc: 92.89%, Val Acc: 92.26%
Epoch [19/20] Loss: 0.2481, Train Acc: 93.08%, Val Acc: 92.62%
Epoch [20/20] Loss: 0.2428, Train Acc: 93.22%, Val Acc: 92.59%
```



최종 Test Accuracy: 93.43%

4. 활성화 함수 relu vs sigmoid

function	최종 loss	최종 val acc	total acc	comment (성능 평가)
sigmoid	0.4172	88.49	89.45	하
relu	0.2511	92.83	93.15	상

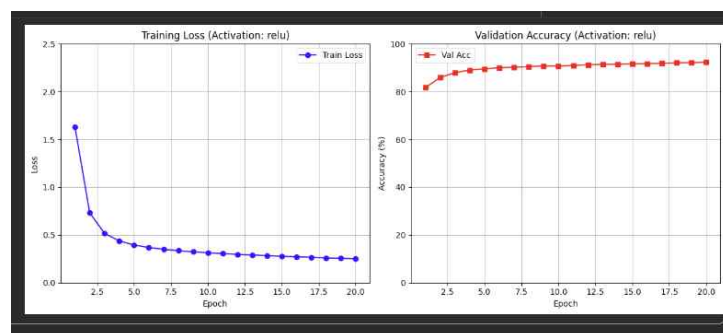
본 실험을 통해 활성화 함수 선택이 모델 수렴 속도에 미치는 결정적인 영향을 확인하였다. Sigmoid는 출력값이 양극단으로 갈수록 기울기가 0에 수렴하는 기울기 소실(Gradient Vanishing) 문제로 인해 학습이 매우 느리게 진행되었다. 반면, ReLU는 양수 영역에서 기울기가 1로 유지되어 역전파 시 신호를 효과적으로 전달함으로써 훨씬 빠르고 안정적인 학습을 가능케 하였다. 따라서 현대 심층 신경망 설계 시 ReLU가 표준적으로 사용되는 이유를 실증적으로 이해할 수 있었다.

4.1. relu

```
*** Epoch [1/20] Loss: 1.6482, Val Acc: 81.34%
Epoch [2/20] Loss: 0.7354, Val Acc: 86.42%
Epoch [3/20] Loss: 0.5180, Val Acc: 88.08%
Epoch [4/20] Loss: 0.4384, Val Acc: 88.85%
Epoch [5/20] Loss: 0.3966, Val Acc: 89.60%
Epoch [6/20] Loss: 0.3706, Val Acc: 90.06%
Epoch [7/20] Loss: 0.3519, Val Acc: 90.34%
Epoch [8/20] Loss: 0.3376, Val Acc: 90.61%
Epoch [9/20] Loss: 0.3257, Val Acc: 90.95%
Epoch [10/20] Loss: 0.3156, Val Acc: 91.18%
Epoch [11/20] Loss: 0.3068, Val Acc: 91.39%
Epoch [12/20] Loss: 0.2989, Val Acc: 91.68%
Epoch [13/20] Loss: 0.2913, Val Acc: 91.89%
Epoch [14/20] Loss: 0.2846, Val Acc: 92.03%
Epoch [15/20] Loss: 0.2782, Val Acc: 92.27%
Epoch [16/20] Loss: 0.2722, Val Acc: 92.40%
Epoch [17/20] Loss: 0.2666, Val Acc: 92.57%
Epoch [18/20] Loss: 0.2612, Val Acc: 92.63%
Epoch [19/20] Loss: 0.2559, Val Acc: 92.80%
Epoch [20/20] Loss: 0.2511, Val Acc: 92.83%
```

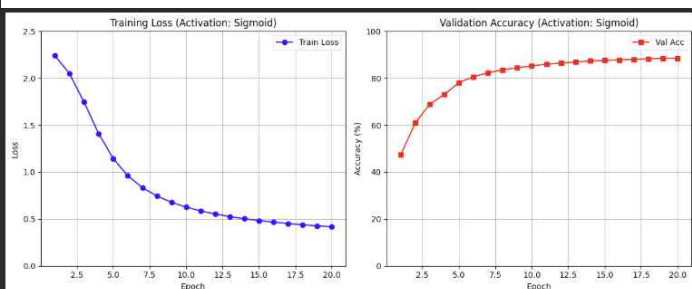
```
# --- [최종 Test Accuracy] --- [cite: 40]
model.eval()
correct = 0
with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        correct += (predicted == labels).sum().item()
print(f'***최종 Test Accuracy: {100 * correct / len(test_dataset):.2f}%')
```

최종 Test Accuracy: 93.15%



4.2. sigmoid

```
Epoch [1/20] Loss: 2.2404, Train Acc: 28.77%, Val Acc: 47.23%
Epoch [2/20] Loss: 2.0480, Train Acc: 56.43%, Val Acc: 60.92%
Epoch [3/20] Loss: 1.7459, Train Acc: 65.63%, Val Acc: 68.93%
Epoch [4/20] Loss: 1.4103, Train Acc: 72.31%, Val Acc: 73.07%
Epoch [5/20] Loss: 1.1445, Train Acc: 76.64%, Val Acc: 78.10%
Epoch [6/20] Loss: 0.9611, Train Acc: 79.67%, Val Acc: 80.66%
Epoch [7/20] Loss: 0.8348, Train Acc: 81.76%, Val Acc: 82.32%
Epoch [8/20] Loss: 0.7447, Train Acc: 83.19%, Val Acc: 83.55%
Epoch [9/20] Loss: 0.6780, Train Acc: 84.33%, Val Acc: 84.49%
Epoch [10/20] Loss: 0.6266, Train Acc: 85.23%, Val Acc: 85.25%
Epoch [11/20] Loss: 0.5860, Train Acc: 85.94%, Val Acc: 85.98%
Epoch [12/20] Loss: 0.5531, Train Acc: 86.53%, Val Acc: 86.49%
Epoch [13/20] Loss: 0.5258, Train Acc: 87.02%, Val Acc: 86.86%
Epoch [14/20] Loss: 0.5028, Train Acc: 87.38%, Val Acc: 87.35%
Epoch [15/20] Loss: 0.4831, Train Acc: 87.69%, Val Acc: 87.60%
Epoch [16/20] Loss: 0.4663, Train Acc: 88.03%, Val Acc: 87.82%
Epoch [17/20] Loss: 0.4516, Train Acc: 88.33%, Val Acc: 88.03%
Epoch [18/20] Loss: 0.4387, Train Acc: 88.55%, Val Acc: 88.28%
Epoch [19/20] Loss: 0.4273, Train Acc: 88.75%, Val Acc: 88.47%
Epoch [20/20] Loss: 0.4172, Train Acc: 88.96%, Val Acc: 88.49%
```



최종 Test Accuracy: 89.45%

5. 전체 고찰

본 실험에서는 별도의 random seed를 사용하지 않고 매 실행시 발생하는 무작위성을 허용하였다. 이로 인해 동일 조건에서의 실험도 수치가 미세하게 변동했으며 딥러닝의 불확실성이라는 특성을 직접 체험했다.

변인 통제를 위해 한 번에 하나의 파라미터만 변경하였으며 모든 최적 조합 결정은 validation set를 기준으로 수행했다.

$lr=0.1$, $batch=1$, $hidden\ node=128$, ReLU 조건에서 가장 높은 성능을 보였다.

파라미터별로 과적합/과소적합의 결과를 관찰할 수 있었다.

6. 질문에 대한 답

6.1. Learning rate가 너무 작거나 너무 클 때 어떤 현상이 나타났는가?

너무 작을 때는 가중치 업데이트가 미미해 학습 속도가 느리고, 20 Epoch 내에 학습을 제대로 완료하지 못하는 과소적합(Underfitting) 현상이 나타났다.

너무 클 때는 일반적인 경우 최적점을 지나쳐 오버슈팅될 위험이 있지만, 본 실험에서는 의외로 가장 높은 성능을 보였다. 비교적 친절한 성향을 띠는 학습 데이터의 영향이라고 예상된다.

6.2. Batch size에 따라 학습 안정성이나 성능이 어떻게 달라졌는가?

batch size가 작을 때에는 업데이트 횟수가 많아 정확도가 매우 높게 나타났지만, 학습 시간이 매우 오래 걸린다는 치명적인 단점이 있었다. 반대로 batch size가 클 때에는 그래프가 매끄럽고 속도가 빠르지만 성능이 저하된다는 특징이 있었다.

6.3 . 은닉층 노드 수가 증가할수록 항상 성능이 좋아졌는가?

은닉층 노드 수가 증가한다고 항상 성능이 좋아진 것은 아니다. 32->128에서는 유의미한 정확도 향상이 관찰되었으나 512까지 증가했을 경우 오히려 정체되는 모습을 보였다. 특정 수준 이상의 노드 수가 MNIST 데이터셋에 대해서는 성능 향상에 기여하지 못했다.

6.4. ReLU와 Sigmoid 중 어느 쪽이 더 잘 작동하였는가?

ReLU가 압도적으로 잘 작동했다.

6.5. 가장 성능이 좋았던 하이퍼파라미터 조합은 무엇인가?

Learnig Rate = 0.1 / Batch Size = 1 / Hidden Nodes = 128 / Activation = ReLU

6.6. train 성능과 validation 성능 차이를 통해 과적합 또는 과소적합을 확인할 수 있었는가?

$lr=0.0001$ 이나 sigmoid에서 성능이 낮게 정체되는 것을 확인했다.

6.7. 본인이 생각하는 가장 중요한 하이퍼파라미터는 무엇이며, 그 이유는 무엇인가?

Learning Rate 가 가장 중요하다고 생각한다. 다른 파라미터가 최적이라 할지라도 학습률이 너무 낮으면 학습 자체가 이루어지지 않고 너무 높으면 발산될 우려가 있기 때문에, 모델이나 데이터셋의 특성에 따라 적절한 학습률을 조절하는 것이 매우 중요하다 .